PlaceOS 3D 디지털 트윈 진행 정리
+ 공실 분석(층수 · 이동시간 · 구역 인구) 점검

작성 2026-10-08 · 저장소 dlrlfwo73123-lab/SpaceOS---Page · 브랜치 claude/bold-goldberg-yae5dr (P0~P6 + QA)

먼저 읽어주세요 — 이 문서의 근거 범위

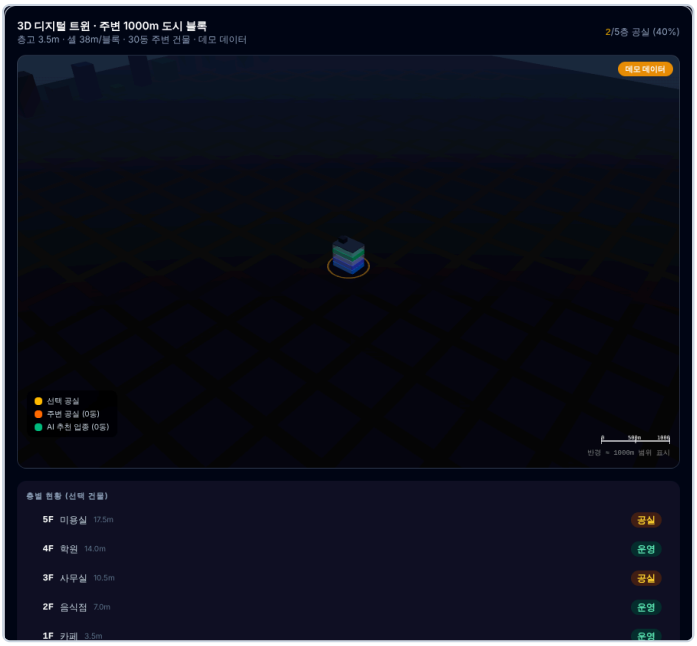
- **https://placeos.web.app/ 에는 이 작업 환경에서 접속할 수 없었습니다**(네트워크 정책 403). 그래서 라이브 사이트를 직접 조작해 본 결과가 **아닙니다**. 근거는 ① 저장소 코드(앱·백엔드), ② 같은 코드를 로컬에서 실행해 찍은 화면, ③ 코드 기반 재현 실험입니다.
- 라이브 사이트가 이 저장소의 어느 빌드인지(배포 방식·백엔드 유무)는 확인하지 못했습니다. 아래 "직접 확인하는 방법"으로 확인해 주세요.
- 화면에 나오는 건물·공실·인구 수치는 모두 **합성(시드 기반) 데이터**이며 앱 자신도 "데모 데이터"라고 표시합니다. 실제 서울 데이터로는 아직 생성·검증하지 못했습니다.

0. 한눈에 보는 결론

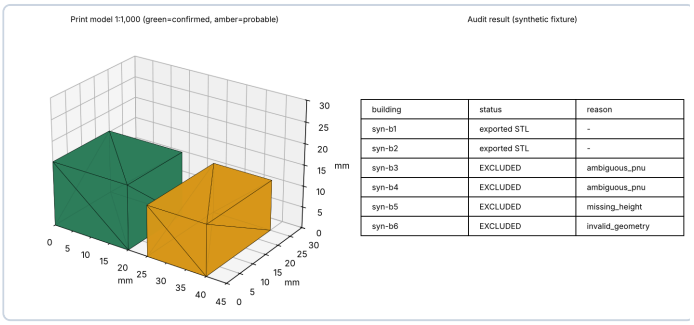
질문	결론
① 히트맵에서 공실 선택 시 층수 결과가 정확한가	정확하지 않음 지도 팝업/모달의 "층"과 3D 트윈의 층별 공실이 **서로 다른 가짜 값에서 따로 나옵니다**. 코드 재현 실험(공실 점 498개, 25개 구)에서 팝업이 말한 층이 트윈에서도 공실인 경우는 백엔드 연결 시 **25.7%**, 백엔드가 없을 때 **37.6%**뿐이고, 백엔드 연결 시 3.4%는 그 층이 건물에 아예 없습니다. 모달·데이터 신뢰성 패널도 "데모 데이터 — 실제 매물과 다름"이라고 밝힙니다.
② 대중교통·차량 소요시간 분석	현재 기능 없음 코드에 이동시간 기능이 없습니다. 방법: 자동차는 네이버 클라우드 Directions 5/15(자동차 전용), 대중교통은 별도 API(ODsay LAB 또는 TMAP 등, 조건은 확인 필요) + 도보 접근 시간 + 시간대 비교. 3장에 절차·호출량·주의사항 정리.
③ 공실 구역의 요일·시간대별 인구	방법 확정, 실데이터 미연동 현재 "유동인구"는 동 기준 단일 합성 수치입니다. 서울 열린데이터광장 생활인구(행정동 단위 제공 중, 집계구 단위는 2026-07-31로 생산 종료 → 250m 격자)를 쓰면 요일×시간대 분석이 가능하며, **요일×시간 집계 코드를 구현·테스트(3개)**하고 예시 결과를 4장에 실었습니다.
3D 디지털 트윈 과정	P0~P6 완료(합성/로컬 검증). 바닥(차도·보도·횡단보도) · 층 슬래브 certainty 색 · 구 경계/공실률 · 건물 외곽 입체 · 출력용 3MF · 실데이터 검증기·CI. 1장에 사진과 함께 정리.

1. 3D 디지털 트윈 진행 과정 (사진 포함)

단계	내용	검증 결과	한계
PDF 보고서 기반 출력 PoC	`tools/print3d` : 감사·좌표/축척·압출·메시 검사·STL·개념 배치·증거 게이트	PDF 제안 테스트 11개 구현·통과	실데이터(Gold) 없이 합성 샘플, 슬라이서·프린터 미사용
P1 바닥	OSM 도로 → 차도·보도·횡단보도 면, 폭 규칙(태그>차선×3m>등급 기본값), 추정 표시, 타일 메시, 트윈 렌더(성동구 시범)	타일 면적 보존 1% 이내, 타입·빌드 통과	Overpass 접속 불가 → 실데이터 미생성
P2 슬래브·구	층 certainty 색(확정 빨강/추정 주황/그 외 회색), 구 경계선(공실률 색), 구 공실률 표시, 공실 바닥 데칼	백엔드 테스트 포함, 구간 판정 불일치 버그 발견·수정	certainty·공실 위치는 placeholder
P3 서울 확장	25개 구 프리셋, 조각 내려받기·병합, 서울 한 벌 타일, 성능 예산 정의	합성 고밀도 1km 열기 0.21MB, 삼각형 2만	실기기 프레임 미측정
P4 건물	OSM 건물 외곽 입체(층 줄무늬), 높이 추정 구분, 성능 보호 장치(fps 낮으면 반경 축소)	고밀도에서 600m 3.5fps(소프트웨어) → 보호 장치로 8.7fps	실데이터 높이 추정 비율 미확인, 라이선스 미결정
P5 출력	트윈 데이터 → 1:1,000 색 파트 3MF/STL, 장비 한계 필수 검수	테스트 10개, 3MF 왕복, 결함(음수 체적) 발견·수정	슬라이서 호환·실출력 미검증
P6 투입 준비	실행기·타일 검증기(예산 게이트)·CI, Python 3.11 새 환경 69개 통과	손상 8종 탐지 테스트	GitHub Actions 실제 실행 미확인

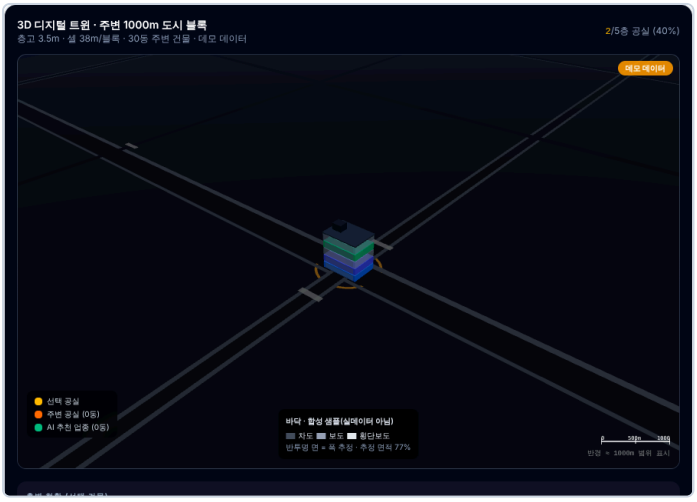


초기: 층별 현황이 있는 3D 트윈(합성 층 데이터, 지도 SDK 차단으로 컴포넌트 직접 마운트)

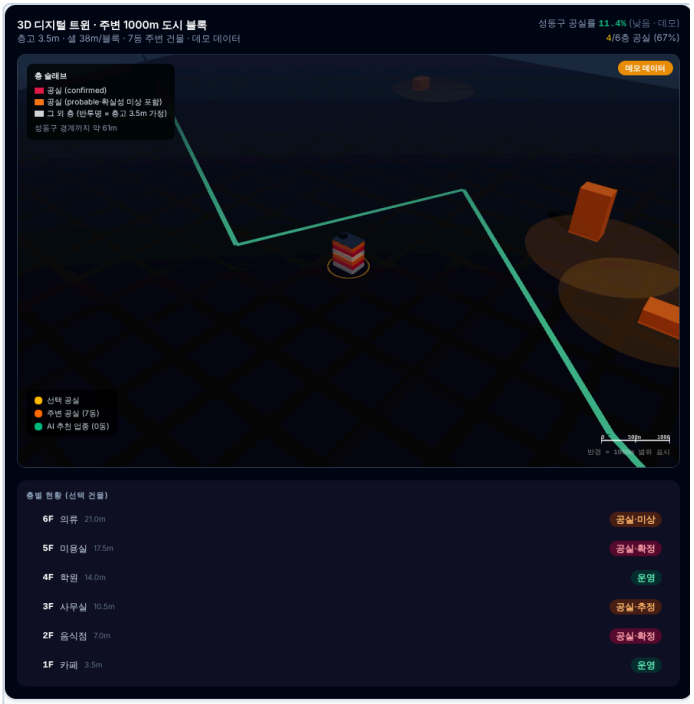


building	status	reason
syn-b1	exported STL	-
syn-b2	exported STL	-
syn-b3	EXCLUDED	ambiguous_pnu
syn-b4	EXCLUDED	ambiguous_pnu
syn-b5	EXCLUDED	missing_height
syn-b6	EXCLUDED	invalid_geometry

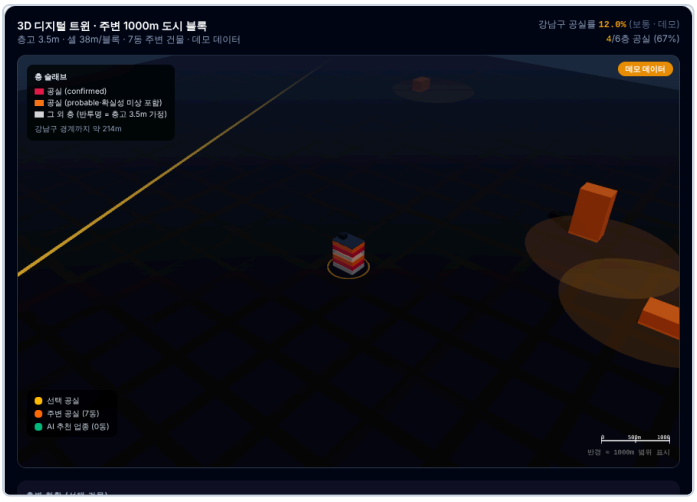
PDF 기반 출력 PoC: 1:1,000 모형과 감사 결과(제외 4건·사유)



P1: 차도·보도·횡단보도 바닥 레이어(합성 샘플)



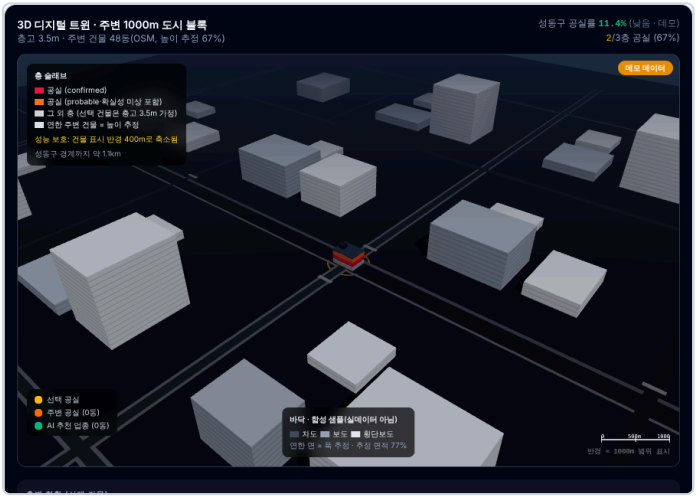
P2: 층 슬래브 색, 구 경계(공실률 색), 구 공실률, 공실 데칼



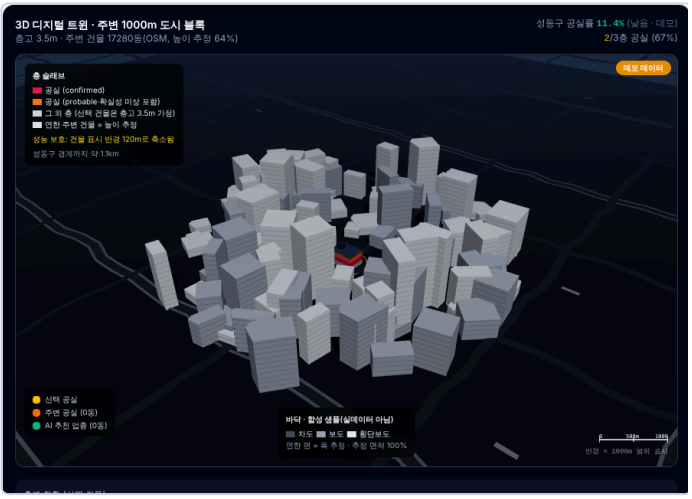
P2: 강남구(12.0%, 보통=주황 경계선)



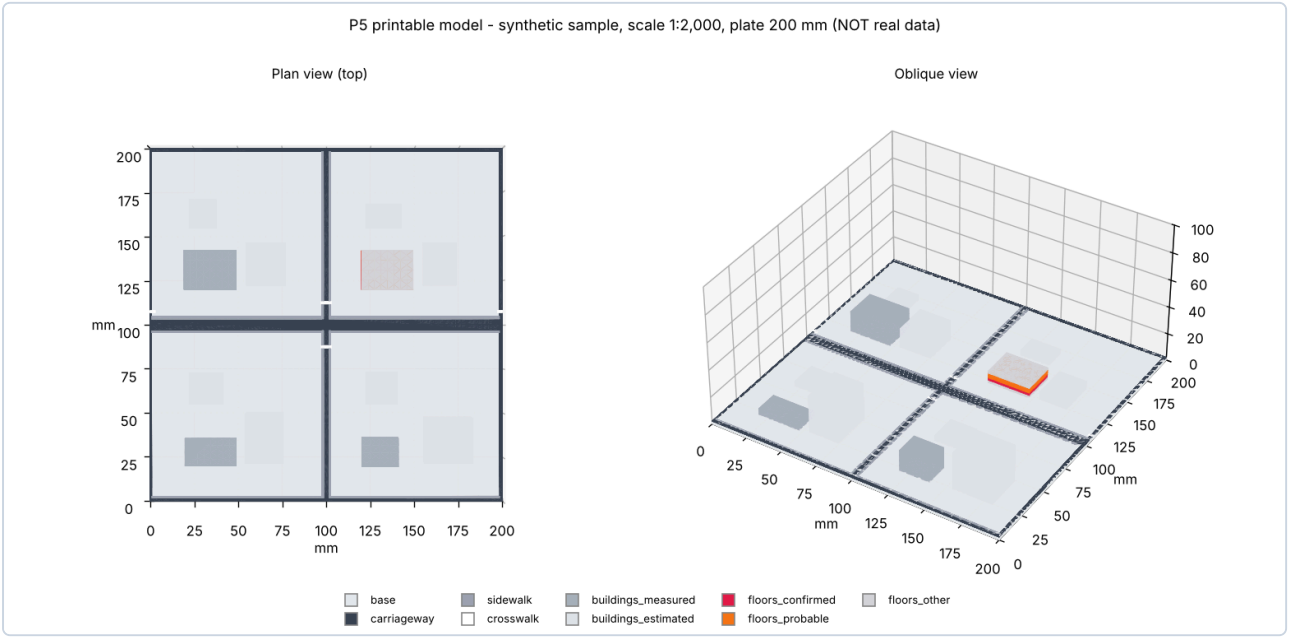
P3: 도심 밀도 합성 도로망(고밀도 바닥 렌더)



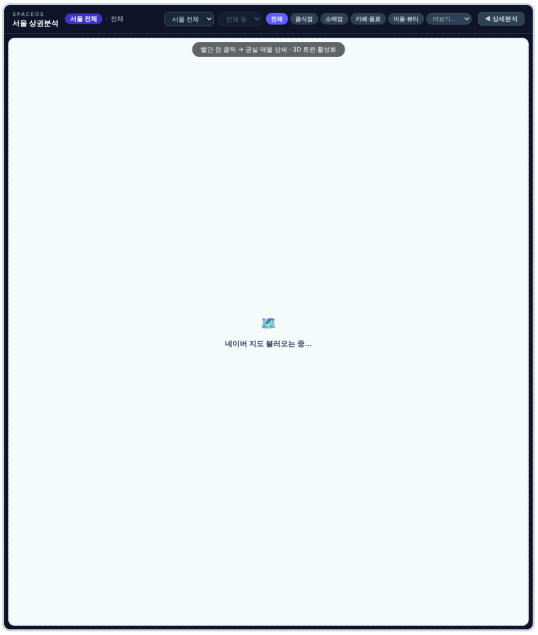
P4: OSM 건물 외곽 입체와 층 줄무늬(합성 샘플 48동)



P4: 17,280동 합성 도심 — 성능 보호로 표시 반경 120m



P5: 출력용 색 파트 모형 미리보기(합성 샘플, 1:2,000, 판 200mm). 선택 건물 층 슬래브가 빨강/주황/회색.



참고: 이 환경에서는 네이버 지도 SDK가 차단되어 앱 기본 화면이 "지도 불러오는 중..."에서 멈춤(앱이 죽지는 않음). 지도에서 점을 눌러 트윈으로 들어가는 흐름은 이 환경에서 검증하지 못했습니다.

2. 질문 ① 히트맵에서 공실을 선택하면 층수 결과가 정확하게 나오는가

2.1 앱에서 실제로 일어나는 일 (코드 근거)

1. 지도(`NaverMap.tsx`)에는 **동 마커(공실률에 따라 색)**와 **빨간 점(공실 매물)**이 찍힙니다. 구를 선택하면 `getVacancyMarkers()` 가 공실 점 15개를 만듭니다. Mapbox 히트맵 컴포넌트(`VacancyHeatmap.tsx`)는 저장소에 있지만 **App에서 사용되지 않습니다**(코드 검색 결과) — "히트맵"은 동 마커 색 + 빨간 점으로 보입니다. 라이브 화면에서 확인이 필요합니다.
2. 빨간 점을 누르면 팝업과 모달에 **"층: N층"**이 표시됩니다. 이 값은 `marker.floor` 이며 **시드 난수로 만든 1~5 사이 값**입니다 (`marketData.getVacancyMarkers`).
3. 같은 클릭이 3D 트윈을 엽니다: `onSelectBuilding('vacancy-<id>')` → `BuildingTwin` 이 `/api/v1/buildings/vacancy-<id>/floors` 를 호출해 **층별 현황**을 그립니다.
4. 백엔드의 `get_building_floors` 는 건물 ID 문자열로 만든 **결정적 가짜 값**(4~8층, 약 25%가 공실)을 돌려줍니다. 백엔드가 없으면(정적 호스팅) 프런트가 **고정 5층(2·4층 공실)**으로 대체합니다.
5. 정리하면 2번의 **팝업 층**과 4번의 **트윈 층별 현황**은 **서로 연결되어 있지 않습니다**(각자 다른 가짜 값).

2.2 재현 실험 (`tools/qa/floor_consistency.py` , 외부 접속 없음)

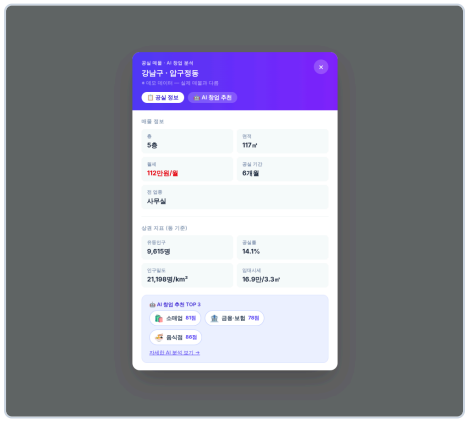
앱이 구를 선택했을 때 만드는 공실 점을 25개 구 전체에 대해 그대로 생성하고(498개), 팝업의 층이 트윈에서도 공실로 나오는지 비교했습니다.

시나리오	팝업 층이 건물에 존재	팝업 층이 트윈에서도 공실
A. 백엔드 연결(가짜 층 데이터)	96.6% (나머지 3.4%는 층 자체가 없음)	**25.7%** (128/498)
B. 백엔드 없음(고정 5층, 2·4층 공실)	100%	**37.6%** (187/498)

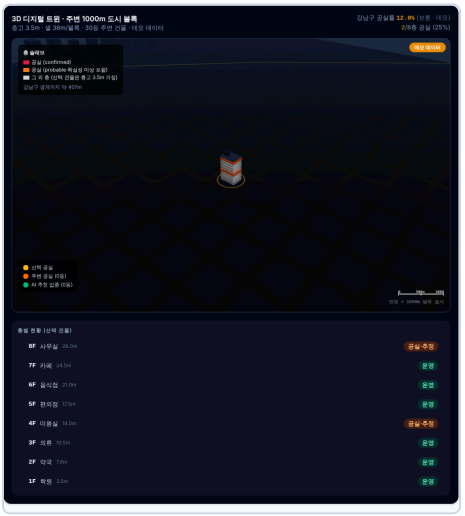
무작위로 맞는 수준이라는 뜻입니다(팝업 층 1~5 중 공실 층이 걸릴 확률 정도). 불일치 예:

공실 점 ID	팝업/모달의 층	트윈 건물	트윈의 공실 층
11680101-v0	5층	8층 건물	4, 8층
11680104-v0	5층	6층 건물	4층
11680107-v0	2층	4층 건물	4층
11680110-v0	2층	8층 건물	1, 5층
11680116-v0	3층	4층 건물	1층
11680122-v0	4층	6층 건물	2, 6층

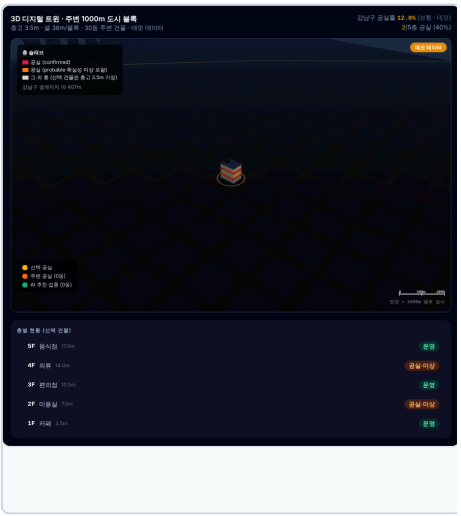
2.3 화면 증거 (같은 공실 점: 강남구 압구정동, ID 11680101-v0)



모달: **5층**, "※ 데모 데이터 — 실제 매물과 다름"



같은 점의 트윈(백엔드 연결): **8층 건물**, 공실은 **4F·8F** — 5층은 운영 중



백엔드가 없을 때(정적 배포 가정): 고정 **5층**, 공실 2F·4F — 모달 5층과 또 불일치



앱의 "데이터 신뢰성 현황": **공실 매물 위치 = 데모 10%**("현재 지도의 빨간 점은 시드 기반 합성 위치, 실제 공실 좌표가 아님"), 유동인구 62%(합성 보정), 전체 평균 48%.

2.4 라이브 사이트에서 직접 확인하는 방법 (5분)

1. placeos.web.app → 구 선택 → **빨간 점** 클릭 → 팝업의 **층**을 적어 둡니다.
2. 오른쪽 ◀ **상세분석**에서 3D 트윈의 층별 현황을 열어, 방금 적은 층이 **공실**인지, 건물 층수가 그 층을 포함하는지 확인합니다.
3. 브라우저 개발자 도구(F12) → Network에서 `/api/v1/buildings/vacancy-…/floors` 요청을 찾습니다. **200 + JSON**이면 백엔드가 있는 것이고, **404/HTML 응답/실패**면 앱이 고정 5층(2·4층 공실)으로 대체한 것입니다. 같은 점을 새로고침해도 결과가 같다면 시드 기반 합성값입니다.
4. 점을 5~10개 눌러 "팝업 층이 트윈에서도 공실"인 비율을 세어 보면 위 실험 수치(약 25~38%)와 비슷하게 나옵니다.

2.5 정확하게 만들려면

- **한 건물·한 층에 대한 단일 데이터 원천**: `building_id, floor, status(vacant/occupied/unknown), certainty(confirmed/probable), source, as_of` . 지도 팝업·모달·트윈이 이 한 레코드를 함께 사용(트윈 P2의 certainty 색과 같은 규칙). 지금은 세 군데가 각자 가짜 값을 만듭니다.
- **건축물대장은 "층·용도·면적"을 알려줄 뿐 공실 여부는 알려주지 않습니다.** 공실 판정에는 층별 점포/임대 정보와의 결합이 필요합니다(예: 상가업소 정보의 층 필드, 임대차·부동산 매물 데이터). 어떤 데이터셋이 층 필드를 갖는지는 **데이터 정의서로 확인해야 합니다**(여기서는 확인하지 못함).
- **불일치 회귀 테스트**: `tools/qa/floor_consistency.py` 를 "팝업 층 = 트윈에서 공실인 층 100%"를 요구하는 테스트로 바꿔 CI에 넣으면 같은 문제가 재발하지 않습니다.
- **임시 대책(빠름)**: 공실 점의 `floor` 를 트윈에 넘겨 해당 층을 공실로 표시하고 건물 층수를 그 층 이상으로 보장 — 단, 이것은 일관성만 맞출 뿐 **정확성(실제 층)은 아닙니다**. 지금은 구현하지 않았습니다(요청 범위 밖).

3. 질문 ② 공실 위치 파악 후 대중교통·자차 소요시간 분석

3.1 현재 상태

코드 검색(`대중교통, 길찾기, directions, transit, 소요시간` 등)에서 **이동시간 기능은 없습니다**(관련 호출·UI·백엔드 모두 없음). 공실 점의 좌표(`lat, lng`)는 있으므로(합성 좌표) 좌표를 입력으로 쓰는 분석을 붙이는 구조입니다. 실제 공실 좌표는 아직 연동되지 않았습니다(앞 장 신뢰성 패널).

3.2 무엇을 측정할지 먼저 정하기

항목	제안
출발지	주요 거점(가까운 지하철역 3곳, 업무지구, 주거 밀집지 등) 또는 "고객 유입권" 대표점. 공실마다 같은 출발지 세트를 써야 비교 가능
수단	도보 · 대중교통(버스+지하철) · 승용차
시간대	평일 출근(08시), 점심(12시), 퇴근(18시), 심야(22시), 주말 오후(14시)
지표	총 소요시간, 환승 횟수, 도보 구간 길이, (자차) 통행료·거리, 시간대별 편차

3.3 방법 A — 키 없이 수동 점검 (소규모·검증용)

공실 좌표를 지도 앱(네이버지도·카카오맵·티맵)의 목적지로 넣고 출발지와 **출발 시각을 바꿔** 길찾기를 눌러 소요시간을 기록합니다. 10~20곳 정도를 비교하거나, API 결과가 맞는지 교차 확인할 때 쓰기 좋습니다. 현재 모달에는 좌표가 표시되지 않으므로, 점검용으로 모달에 위도·경도(복사 버튼)를 추가하는 것을 권장합니다.

3.4 방법 B — API로 자동 계산

구분	내용	확인 상태
자차 네이버 클라우드 Directions 5 / 15	출발지·목적지(경도,위도)로 **자동차 경로**의 소요시간·거리·통행료·유류비 조회. 경유지 최대 5개(5) / 15개(15). **실시간 교통을 반영**하므로 같은 입력이라도 결과가 달라질 수 있음. 요청 경로: Directions 5 `map-direction/v1`, 15 `map-direction-15/v1`. API Gateway 인증 필요(Client ID/Secret 헤더).	공식 문서로 **확인됨**: 자동차 전용, 실시간 교통, 경유지 수, 인증 필요, 월 상한 표기 5,000,000건. **미확인**: 단가·무료 제공량(공식 표가 비어 있음), 임의 출발시각 지정 지원 여부
대중교통 ODsay LAB / TMAP 등	Directions는 **자동차 전용**이라 대중교통은 별도 API가 필요. ODsay는 환승 경로·소요시간 조회용으로 알려져 있고 서버 키는 호출 IP 화이트리스트가 필수. TMAP 대중교통 API도 후보.	**미확인**: 엔드포인트·응답 필드·요금·일 호출 한도·출발시각 지정. 두 서비스에 같은 출발/도착으로 **샘플 호출을 해 비교**한 뒤 선택 권장
도보 접근	가까운 지하철역·정류장까지 직선거리 ÷ 80m/분(보행 속도 가정)으로 근사, 정밀값은 도보 길찾기 API	역·정류장 좌표 데이터셋은 서울 열린데이터광장/공공데이터포털에서 확인 필요

```
# 자차 요청 형식 예시(공식 문서에서 최신 URL·헤더명을 확인한 뒤 사용; 키는 서버 환경변수에만 보관)
GET https://maps.apigw.ntruss.com/map-direction/v1/driving?start=<경도>,<위도>&goal=<경도>,<위도>
헤더: 인증용 Client ID / Client Secret (네이버 클라우드 콘솔에서 발급)
응답에서 확인할 값: route.<옵션>[0].summary.duration(밀리초), distance(m), tollFare, fuelPrice
```

3.5 앱에 넣는 설계

- **키 보호**: 브라우저에서 직접 호출하지 말고 **백엔드 프록시**(`/api/v1/accessibility?lat=&lng=`)로 호출 — `VITE_` 변수는 브라우저에 노출됩니다.
- **호출량 계산**: 호출 수 = 공실 수 × 출발지 수 × 시간대 수 × 수단 수. 예) 공실 500 × 출발지 5 × 시간대 5 × 수단 2 = 25,000건. 공실 선택 시 즉시 호출하지 말고 **미리 계산해 캐시**(공실 좌표가 같으면 재사용)하고 갱신 주기를 정합니다.
- **결과 표시**: 출발지별 소요시간 막대, 시간대별 변화, "대중교통/자차 비율", 가장 가까운 역까지 도보시간. 값에는 **산출 시각·API 출처**를 함께 표기.
- **주의**: 실시간 교통은 "지금" 기준이라 **과거/미래의 특정 요일·시간대 값이 아닙니다**. 시간대별 대표값이 필요하면 주기적으로 호출해 직접 누적하거나, 출발시각 지정을 지원하는 API를 확인해야 합니다.

4. 질문 ③ 공실 구역의 요일별·시간대별 인구 분석

4.1 현재 상태

모달의 **유동인구**는 "동 기준" 단일 수치(예: 9,615명)이며 시드 기반 합성값입니다(신뢰도 62%로 표시). **요일·시간대 구분이 없고** 공실 위치가 속한 좁은 구역 단위도 아닙니다. 신뢰성 패널이 "시간대별 실제 유동인구 조회"를 **향후 개선 방안으로만** 언급합니다(코드 검색에서 요일·시간대 기능은 확인되지 않음).

4.2 쓸 수 있는 데이터 (검색으로 확인한 범위)

데이터	내용	상태
서울 생활인구(내국인) **행정동 단위** 열린데이터광장 OA-14991	통신 데이터로 추정한 특정 시점·지역의 존재 인구. 월별 CSV(`LOCAL_PEOPLE_DONG_YYYYMM`), 갱신 주기 매일. Sheet/Open API는 최근 2개월만 제공	제공 중 시간대 필드는 CSV 명세 확인 필요
서울 생활인구 **집계구 단위**	통계청 집계구 기준. 2017-01-01~2026-07-31 아카이브로 제공	2026-07-31 생산 종료 250m 국가표준격자로 전환
[내국인] 행정동별 / 자치구별 서울 생활인구(**250m**)	격자 기반. 공실 주변을 250m 셀 단위로 보려면 이쪽이 현실적 대안	안내 확인, 컬럼·격자 ID 규칙은 정의서 확인 필요
자치구 일별 집계표(OA-23031)	총·내국인·외국인 생활인구, 일 최대/최소, 주간(09~18시)/야간(19~08시) 인구	확인됨. 구 단위라 공실 구역 분석에는 거침
상권분석서비스 "길단위인구" 등	상권 배후지 인구 계열. 추정유동인구의 요일·시간대 구간(예: 6구간)이 제공되는지는 **확인하지 못함**	미확인 데이터셋 상세 페이지의 API 명세 확인 필요

4.3 "공실 내 구역"을 어떻게 정의할지

정의	장점	한계
(a) 공실이 속한 **행정동**	지금 바로 가능(제공 중), 좌표→동 매핑만 필요	동이 넓어 골목 단위 차이를 못 봄
(b) 공실이 속한 **250m 격자 셀**	도보 3~4분 생활권에 가까움	격자 ID 규칙·좌표계(국가표준격자) 확인 필요, 외곽·이동 중 인구 포함
(c) 공실 **반경 N m 안의 셀 합**	경계 효과 완화, 상권 규모 비교에 적합	중복 집계·가중 방식 설계 필요

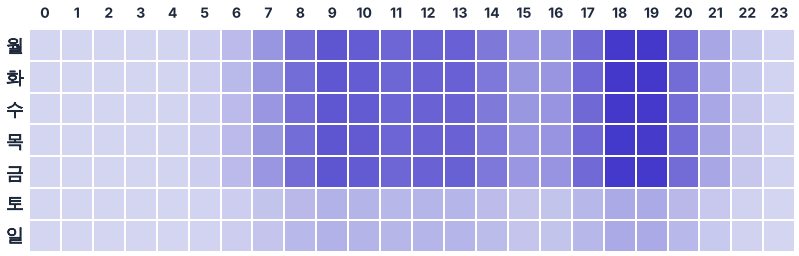
4.4 분석 절차 (구현된 부분 표시)

1. 공실 좌표 → 구역 매핑(행정동 코드 또는 250m 셀) *(미구현: 경계/격자 데이터 필요)*
2. 최근 8주 이상 데이터 수집(공휴일·이벤트 주간 표시)
3. **요일 × 시간대 평균** 집계 — 구현·테스트됨 `tools/qa/population_pivot.py` : 같은 (날짜, 시간)의 여러 구역 값은 먼저 합산한 뒤 같은 요일의 날짜들을 평균. 6개 시간대 구간(00-06, 06-11, 11-14, 14-17, 17-21, 21-24)과 피크, 평일/주말 비율 계산
4. 해석: 피크 요일·시간, 점심/저녁 의존도, 주말 비율 → 업종 적합도(카페/음식점/학원 등)와 연결

```
import population_pivot as pp
table = pp.pivot_weekday_hour(rows, zones=["<행정동코드 또는 셀ID>"])   # 7(월~일) x 24 평균표
print(pp.band_table(table), pp.peak(table), pp.weekday_vs_weekend(table))
```

4.5 결과 예시 (합성 패턴 — 실제 데이터 아님, 출력 형태를 보여주기 위한 것)

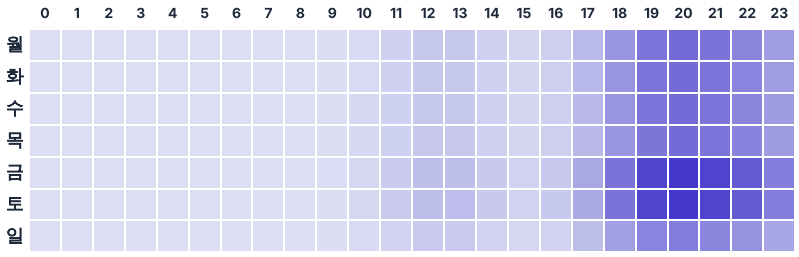
유형 A — 오피스형(평일 출퇴근 피크) · 합성 예시



	00-06	06-11	11-14	14-17	17-21	21-24
월	1,270	4,675	5,668	4,073	6,336	2,032
화	1,270	4,665	5,688	4,058	6,351	2,025
수	1,270	4,665	5,682	4,058	6,343	2,029
목	1,270	4,675	5,659	4,073	6,323	2,038
금	1,270	4,680	5,653	4,082	6,321	2,038
토	1,220	2,173	2,453	2,004	2,640	1,431
일	1,219	2,169	2,460	1,997	2,644	1,428

피크 수요일 18시 (7,345명) · 평일 평균 3,818 / 주말 평균 1,933 (주말/평일 0.51)

유형 B — 먹자골목형(저녁·주말 피크) · 합성 예시



	00-06	06-11	11-14	14-17	17-21	21-24
월	1,500	1,586	2,808	2,352	7,087	7,402
화	1,500	1,583	2,817	2,343	7,097	7,384
수	1,500	1,583	2,814	2,344	7,079	7,403
목	1,500	1,584	2,802	2,354	7,060	7,432
금	1,500	1,624	3,386	2,741	9,575	10,093
토	1,500	1,623	3,399	2,733	9,606	10,052
일	1,500	1,570	2,621	2,215	6,256	6,501

피크 토요일 20시 (12,729명) · 평일 평균 3,632 / 주말 평균 3,750 (주말/평일 1.03)

두 유형은 같은 도구로 비교됩니다. A는 평일 출퇴근 피크·주말 급감(점심·직장인 업종 유리), B는 저녁·주말이 높은 상권(주점·음식점 유리). 실제 공실 구역에서는 같은 표가 실데이터로 채워집니다.

4.6 주의

- 생활인구는 통신 기반 **추정 체류 인구**이지 방문객 수·매출이 아닙니다. 상주·유입·통과 인구를 구분하려면 추가 지표가 필요합니다.
- 출처 표기와 이용 조건(공공누리 유형 등)을 서비스 반영 전에 확인하세요. 통계 개편(집계구→250m 격자)으로 **데이터 정의가 바뀔 수 있어** 컬럼명·코드 체계는 데이터 정의서를 기준으로 하세요.
- OpenAPI 키는 서버에만 보관하고, 응답은 일/월 단위로 캐시합니다(최근 2개월만 제공되는 제한 때문에 **직접 누적 저장**이 필요할 수 있음).

5. 권장 순서와 결정이 필요한 것

#	작업	이유
1	**공실·층 단일 데이터 원천**과 불일치 회귀 테스트(2.5)	층 정확도 문제의 근본 원인. 이후 모든 분석의 전제
2	**성동구 실데이터 생성**: `py -3.11 tools/ground/run_all.py --gu seongdong` 후 `ground-run-report.md` 공유	바닥·건물 실데이터 예산 판정 (인터넷 되는 PC 필요)
3	행정동 단위 요일×시간대 인구(4.3-a)부터 연동	제공 중인 데이터로 바로 가능, 집계 코드 완료
4	이동시간: 자차(Directions) → 대중교통 API 샘플 비교 후 선택	키 발급·약관·요금 확인 선행

5	라이선스(OSM ODbL 파생 DB, 네이버 지도 데이터 사용 범위) 결정	`docs/digital-twin/data-licenses.md`

소유자 결정 사항: 공실 데이터의 실제 원천(매물/임대/대장), 이동시간 API 선택과 예산, 구역 정의(동/250m 셀/반경), 출발지 세트와 시간대.

부록. 재현 방법과 파일

```
python tools/qa/floor_consistency.py          # 2장 실험(팝업 층 vs 트윈 층) — Node 필요, 외부 접속 없음
python -m pytest tools/print3d/tests tools/ground/tests tools/qa/tests apps/backend/tests -q
node apps/frontend/scripts/check-extrude.mjs  # 건물 압출 점검
```

- 코드: `apps/frontend/src/components/{NaverMap,VacancyModal,BuildingTwin,DataReliabilityPanel}.tsx`, `lib/marketData.ts`, `apps/backend/app/data/buildings.py`
- 문서: `docs/digital-twin/*.md` (P1~P6, 라이선스), `docs/print3d/*.md`
- 이 문서의 한계: 라이브 사이트 미접속, 외부 API 호출·키 발급 미수행, 외부 데이터셋의 필드·요금은 검색으로 확인된 범위만 기재(미확인 항목은 본문에 표시).